

프로젝트 관리론

Chapter 02 UML

목차

01 UML의 이해

02 유스케이스 다이어그램

03 클래스 다이어그램

04 순차 다이어그램

05 통신 다이어그램

06 활동 다이어그램

07 상태 다이어그램

08 컴포넌트 다이어그램

09 배치 다이어그램

학습목표

- UML 다이어그램을 이해한다.
- UML 다이어그램의 용도를 알아본다.
- 유스케이스 다이어그램을 자세히 알아본다.

01. UML의 이해

■ UML

■ UML의 역할

- 소프트웨어의 전체를 판단할 수 있도록 12개의 다이어그램을 제시
- 시스템이 상호작용하는 측면, 시스템 전체 구조 측면, 컴포넌트 간의 관계 등을 시각적으로 볼 수 있게 나타낸 도면

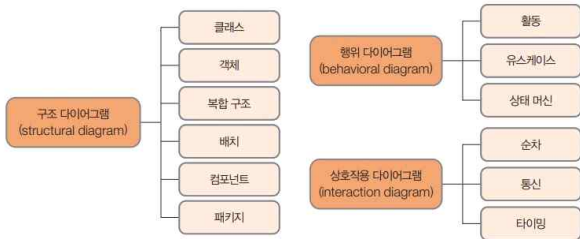


그림 2-1 UML의 12개 다이어그램

02. 유스케이스 다이어그램

■ 액터

■ 액터의 종류

• 사용자 액터

- 사용자 액터는 시스템을 사용하는 사람(역할)을 의미



그림 2-2 액터의 표기

- 액터와 유스케이스의 관계는 화살표(→)를 사용해 표현하며 화살표 방향은 액터에서 유스케이스로 향함



그림 2-3 사용자 액터와 유스케이스의 관계

02. 유스케이스 다이어그램

■ 액터

■ 액터의 종류

• 시스템 액터

- 해당 프로젝트의 개발 범위에는 속하지 않지만 데이터를 주고받는 등 서로 연동되는 또 다른 시스템
- 유스케이스가 시스템 액터를 사용(참조)하는 것이므로 화살표 방향은 유스케이스에서 시스템 액터로 향함



그림 2-4 유스케이스와 시스템 액터의 관계

• 주요 액터

- 시스템에게 작업의 실행을 요구하는 능동적 입장의 액터
- 대부분의 액터가 여기에 해당

• 보조 액터

- 유스케이스로부터 요청을 받거나 메시지를 전달받아 수동적으로 작업을 하는 액터

02. 유스케이스 다이어그램

■ 액터

■ 액터의 종류

• 프록시 액터

- 액터와 시스템의 중간 위치에서 무언가를 대신해주는 액터
- 시스템에 등록할 수 있도록 접근 권한이 부여된 경우에만 프록시 액터가 됨



그림 2-5 프록시 액터의 예

02. 유스케이스 다이어그램

■ 액터

■ 액터 식별

• 액터 식별 방법(학사관리시스템의 예)

- ① 시스템을 사용하는 사람을 찾음
 - 학사관리시스템을 사용하는 사람은 교수, 학생, 조교, 학사담당직원 등
- ② 시스템에 자료를 등록/수정/삭제/조회하는 사람을 찾음
 - 학사관리시스템에서 액터는 성적을 등록/수정/삭제/조회하는 교수, 성적을 조회만 하는 학생, 조교, 학사담당직원
- ③ 시스템에 연동된 또 다른 시스템을 찾음(시스템 액터)
 - 앞에서 학사관리시스템에서 휴학신청 유스케이스는 도서관리 시스템
- ④ 시스템을 유지 및 관리하는 사람을 찾음(보조 액터)
 - 시스템의 기능을 직접 사용하지는 않지만 시스템이 잘 구동되도록 유지 및 관리해주는 유지보수담당자
- ⑤ 유스케이스 기능에 권한은 없지만 역할을 대신하는 사람을 찾음(프록시 액터)

02. 유스케이스 다이어그램

■ 액터

■ 액터 이름

• 액터 이름 설정 규칙

- ① 이름만 봐도 액터의 의미를 알 수 있게 정함
- ② 역할을 나타내는 단어로 사용자 액터의 이름을 정함
 - 부장, 과장 같은 직책이 아닌 역할로 정함
 - 구체적인 역할을 나타내는 단어를 사용
- ③ 시스템 액터는 시스템 이름을 사용해 이름을 정함



그림 2-6 역할을 나타내는 단어를 액터 이름에 사용한 예

02. 유스케이스 다이어그램

■ 액터

■ 액터 목록

- 각 액터에 관한 설명을 붙인 액터 목록을 만들면 그 액터가 시스템을 사용해 어떤 일을 하는지 이해하는 데 도움이 됨

표 2-1 액터 목록

이름	설명
교수	교수는 교과목과 개설과목을 결정하고 성적등록을 한다.
학생	학생은 수강신청, 성적조회, 휴학신청을 한다.
조교	조교는 교과목과 개설과목을 등록(대행)하고 휴학신청 승인을 한다. 조교는 성적장학생 명단을 등록한다.
학사담당직원	학사담당직원은 교과목과 개설과목의 등록 기간을 설정한다.

02. 유스케이스 다이어그램

■ 유스케이스

■ 유스케이스

- 사용자가 시스템을 통해 사용하고 싶은 기능
- 유스케이스가 모여 하나의 서브시스템을 이루고 서브시스템이 모여 개발 시스템이 됨
- 전체 시스템은 유스케이스를 모아 놓은 것과 같아야 함



그림 2-7 유스케이스 예

■ 유스케이스 식별

- ① 사용자인 액터가 시스템에 요구하는 기능을 후보 유스케이스로 선정할 수 있음
 - ② 사용자의 업무에서 유스케이스를 도출할 수 있음
 - ③ 데이터베이스에서 데이터 등록/수정/삭제/조회하는 기능을 하나의 유스케이스로 선정할 수 있음
- 찾은 후보 유스케이스는 여러 번의 정련 과정을 통해 최적화된 유스케이스로 결정
 - 유스케이스를 찾는 작업은 개발자가 하지만 사용자 관점에서 유스케이스를 정의해야 함

02. 유스케이스 다이어그램

■ 유스케이스

■ 유스케이스 식별

- 처음에는 가능하다고 생각되는 것을 모두 도출해 놓고 하나씩 면밀히 살펴보는 과정을 거쳐 불필요한 것은 삭제하고, 필요시 합치거나 분리해 최종 유스케이스를 선정



그림 2-8 학사관리시스템에서 도출된 1차 후보 유스케이스

02. 유스케이스 다이어그램

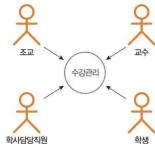
■ 유스케이스

■ 유스케이스 검증

- ① 시스템이 아니라 수작업으로 이루어지는 것은 유스케이스에 포함하면 안 됨
- ② 액터가 원하는 최종 결과만 나타내는 유스케이스인지 확인
- ③ 액터가 수행하는 유스케이스인지 확인
- ④ 유스케이스 내의 이벤트 흐름 전체를 액터가 사용하는지 확인

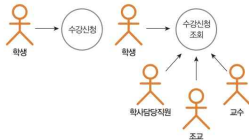


그림 2-9 수작업으로 이루어지는 것은 유스케이스 불가



(a) 부적절한 유스케이스

그림 2-10 유스케이스 검증



(b) 적절한 유스케이스

02. 유스케이스 다이어그램

■ 유스케이스

■ 유스케이스 이름

- ① 사용자 관점에서 이해할 수 있는 이름을 사용



그림 2-11 사용자 관점의 유스케이스 이름

- ② 명사보다는 어떤 기능을 하는지 알 수 있는 동사형 명사를 사용



그림 2-12 동사형 명사를 사용한 이름

- ③ 사용자가 시스템을 통해 얻으려는 최종 목적이 나타나는 이름을 사용

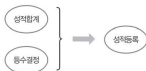


그림 2-13 최종 목적을 사용한 이름

02. 유스케이스 다이어그램

■ 유스케이스

■ 유스케이스 목록

표 2-2 유스케이스 목록

이름	설명
교과목등록	교수는 교과목을 등록/수정/삭제한다.
개설과목등록	교수는 개설과목을 등록/수정/삭제한다.
성적입력	교수는 수강성적을 등록/수정/삭제한다.
강의계획서입력	교수는 강의계획서를 등록/수정/삭제한다.
수강신청	학생은 수강과목을 등록/수정/삭제한다.
개인정보수정	학생은 개인정보를 수정/조회한다.
휴학신청	학생은 휴학을 신청한다.
휴학승인	조교는 학생이 신청한 휴학을 승인한다.
기간설정	학사담당직원은 교과목/개설과목/수강신청에 대한 기간을 설정한다.
출석부조회	교수/조교/학사담당직원은 출석부를 조회한다.
시간표/강의실조회	학생은 시간표/강의실을 조회한다.
성적조회	사용자(교수/학생/조교/학사담당직원)는 성적을 조회한다.
개설과목조회	사용자(교수/학생/조교/학사담당직원)는 개설과목을 조회한다.
교과목조회	사용자(교수/학생/조교/학사담당직원)는 교과목을 조회한다.
학생정보조회	사용자(교수/학생/조교/학사담당직원)는 학생정보를 조회한다.
강의계획서조회	사용자(교수/학생/조교/학사담당직원)는 강의계획서를 조회한다.

02. 유스케이스 다이어그램

■ 관계

■ 액터 → 유스케이스 관계

- 액터와 유스케이스는 연관 관계로 방향성을 갖는 실선으로 표현
- 화살표 방향은 데이터가 흘러가는 방향이 아니라 제어의 흐름을 나타냄
- 제어하는 주체에서 출발해 제어받는 대상으로 연관 방향이 결정



그림 2-14 액터와 유스케이스의 관계

- 시스템 액터도 유스케이스 제어의 주체가 될 수 있음
- 학생(액터)과 증명서자동발급기(시스템 액터) 사이인 액터와 액터 사이에도 연관 관계가 존재



그림 2-15 액터와 시스템 액터, 시스템 액터와 유스케이스 간의 연관 관계

02. 유스케이스 다이어그램

■ 관계

■ 유스케이스 → 액터 관계

- 유스케이스의 수행 결과를 액터에게 알려줄 때는 유스케이스에서 액터로 방향성을 갖는 실선으로 표현
- 유스케이스 → 액터 관계에서 주의할 사항은 통보 기능이 시스템 내에서 이루어져야 함
- 유스케이스 다이어그램에서 화살표로 나타냈다면 그 기능을 개발에 포함해야 함

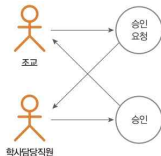


그림 2-16 유스케이스와 액터의 관계

• 유스케이스 → 시스템 액터 관계



그림 2-17 유스케이스와 시스템 액터의 관계

02. 유스케이스 다이어그램

■ 관계

■ 액터의 일반화 관계

- 일반화: 개별적인 것에서 공통적인 특징을 뽑아 이름을 붙인 것(반대 개념: 특수화)

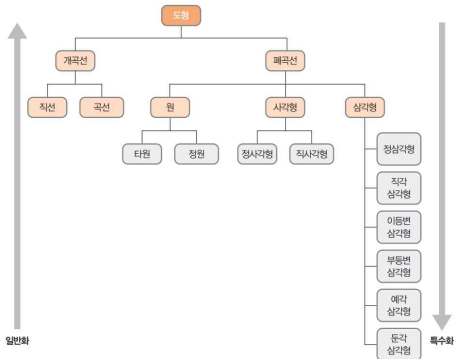


그림 2-18 일반화와 특수화의 관계

02. 유스케이스 다이어그램

■ 관계

■ 액터의 일반화 관계

- 일반화 개념은 액터 사이에 먼저 적용할 수 있음

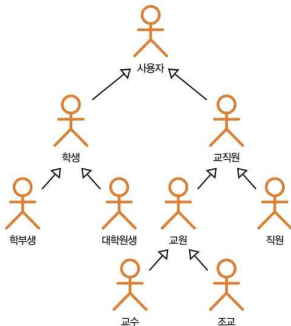


그림 2-19 액터의 일반화 관계

02. 유스케이스 다이어그램

■ 관계

■ 액터의 일반화 관계

- 간결하게 일반화해 복잡함을 줄이면 이해하기가 쉽고 새로운 액터도 간단하게 추가할 수 있음

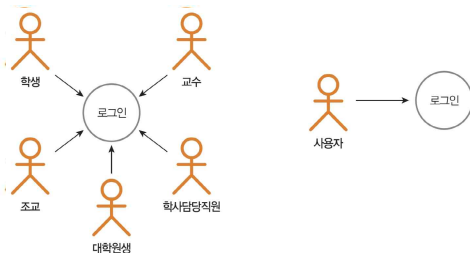


그림 2-20 일반화가 안 된 유스케이스 다이어그램과 일반화가 된 유스케이스 다이어그램

02. 유스케이스 다이어그램

■ 관계

- 액터간의 연관 관계

액터가 또 다른 액터를 통해 유스케이스를 이용하는 경우



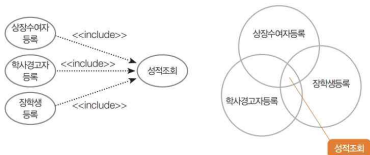
그림 2-21 학생 액터와 조교 액터의 연관 관계

02. 유스케이스 다이어그램

■ 관계

■ 포함 관계

- 유스케이스의 일부 기능이 다른 유스케이스에서도 공통으로 필요하다면 이를 별도로 만든 뒤 호출해 사용하는 것
- 피포함 유스케이스: 공통으로 사용하기 위해 별도 유스케이스로 만들어 놓은 것
- 기본 유스케이스: 피포함 유스케이스를 호출하는 유스케이스
- 포함 관계는 화살표 방향이 기본 유스케이스에서 피포함 유스케이스로 향함
- 점선을 사용하고 스테레오 타입으로 <<include>>라고 표기
- 포함 관계는 화살표 방향이 기본 유스케이스에서 피포함 유스케이스로 향함



(a) 유스케이스 다이어그램으로 나타낸 포함 관계

(b) 벤 다이어그램으로 나타낸 포함 관계

그림 2-22 포함 관계를 나타내는 유스케이스 다이어그램과 벤 다이어그램

- 피포함 유스케이스는 2개 이상의 유스케이스에서 사용할 수 있는 공통 부분(이벤트 흐름)으로 만들어야 함
- 다른 유스케이스에서 사용하지 않는 기능이라면 포함 관계를 만들지 않아야 함

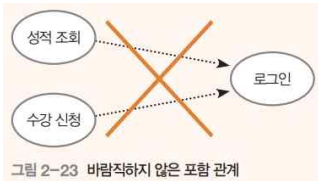
02. 유스케이스 다이어그램

■ 관계

■ 포함 관계

• 포함 관계와 선행 관계

- 포함 관계는 선행 조건(유스케이스 명세서 작성 시 필요)과 혼동될 우려가 있음



- 예) '수강신청' 유스케이스와 '성적조회' 유스케이스에서 공통으로 필요한 부분이 '로그인' 유스케이스라고 혼동

02. 유스케이스 다이어그램

■ 관계

■ 확장 관계

- 특정 조건(메일 도착)이 발생하면 확장 유스케이스(메일 확인)를 수행하고 다시 기존 유스케이스(검색)를 수행
- 확장 관계는 점선 화살표로 표기하고 방향은 기존 유스케이스 쪽으로 향함
- <<extend>>라는 스테레오 타입을 사용



그림 2-24 확장 관계

- 예외적인 상황이나 어쩌다 한 번 발생하는 것을 확장 관계로 표현하는 것은 적절치 않음



그림 2-25 부적절한 확장 관계

03. 클래스 다이어그램

■ 클래스 다이어그램

- 클래스 다이어그램

- 소프트웨어의 기본 구성 단위인 시스템에서 사용하는 클래스를 정의
- 클래스들이 서로 어떻게 연결되어 있고 어떤 역할을 하는지 다이어그램으로 표현

- 클래스

- 데이터(속성)와 메서드를 묶어 놓은 것
- 세 칸의 직사각형 모양
- 첫 번째 칸에는 클래스 이름
- 두 번째 칸에는 클래스의 속성
- 마지막 칸은 클래스가 제공하는 기능인 메서드를 나타냄

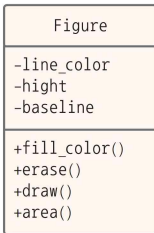


그림 2-26 클래스 표기의 예

03. 클래스 다이어그램

■ 클래스 다이어그램

• 클래스 이름

- 클래스는 다른 클래스와 구별되는 유일한 이름을 가진
- 이름에 명사나 명사구를 사용하며 두 단어를 사용할 때는 붙여 쓰되 각 단어의 첫 글자는 대문자로 씀
- 복수형, 소유격, 형용사는 가급적 쓰지 않음

• 속성

- 클래스가 갖는 정적인 특성
- 속성의 이름은 소문자로 나타내며 두 단어를 사용 할 때는 두 번째 단어의 첫 글자는 대문자로 씀

• 메서드

- 클래스가 외부의 다른 객체에게 제공할 서비스와 기능
- 외부 클래스는 메서드를 통해 해당 클래스에 접근할 수 있음
- 외부에서 이 기능을 요구하는지에 따라 메서드로 도출할지 판단

• 가시성

- 속성과 메서드의 접근 권한을 지정하는 방식

표 2-3 가시성

가시성	설명
공개(public, +)	같은 시스템에 있는 모든 클래스에 접근할 수 있다.
은닉(private, -)	같은 시스템 내의 다른 클래스는 직접 접근할 수 없고 해당 클래스의 메서드를 통해서만 접근할 수 있다. 클래스에서 대부분의 속성은 private으로 설정한다.
부분 공개(protected, #)	다른 클래스가 접근할 수 없고 해당 클래스의 메서드와 클래스를 상속받은 하위 클래스만 접근할 수 있다.

03. 클래스 다이어그램

■ 클래스 다이어그램

- 클래스 다이어그램의 예

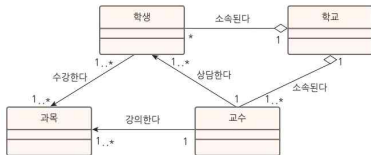


그림 2-27 클래스 다이어그램의 예

- 학생은 여러 과목을 수강할 수 있음
- 학생은 하나의 학교에 소속되어 있음
- 교수는 하나의 학교에 소속되어 있음
- 교수 한 명은 여러 과목을 강의
- 교수 한 명은 여러 명의 학생을 상담

04. 순차 다이어그램

■ 순차 다이어그램

■ 순차 다이어그램

- 실행 시점에 객체들이 어떻게 상호 동작하는지를 메시지 순서에 초점을 맞춰 나타낸 것
- 어떠한 작업이 객체 간에 발생하는지를 시간 순서에 따라 쉽게 파악할 수 있음

■ 순차 다이어그램의 구성요소

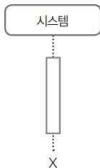
• 객체

- 순차 다이어그램에서 객체는 메시지를 보내고 받는 것
- '객체명: 클래스명'으로 나타내며 한쪽은 생략할 수 있음

객체명: 클래스명

• 객체 생명선

- 객체의 생존 기간을 나타내며 X 표시는 객체가 소멸하는 시점
- 위에서 아래로 내려가는 점선으로 나타냄
- 생명선을 따라 나타나는 직사각형은 활성화 구간으로 객체의 메서드가 실행되고 있음을 나타냄
- 구간의 길이는 메서드 의 실행 시간을 나타냄
- 활성화 구간은 반드시 존재하는 것은 아니며 생략 가능



04. 순차 다이어그램

■ 순차 다이어그램

■ 순차 다이어그램의 구성요소

• 메시지

- 객체와 객체의 상호작용을 표현하는 것으로 화살표를 이용
- 화살표 위에는 수신 객체의 함수명을 명기

• 동기 메시지

- 송신 객체가 수신 객체에 서비스를 요청(메서드 호출)하면 메서드 실행이 완료될 때까지 기다림
- 실선과 속이 채워진 삼각형으로 나타냄



• 비동기 메시지

- 송신 객체가 수신 객체에 서비스를 요청(메서드 호출)할 때 메서드의 실행과 상관없이 다음 작업을 수행
- 수신 객체로부터의 반환도 기다리지 않는다.
- 일반 화살표 모양으로 나타냄



04. 순차 다이어그램

■ 순차 다이어그램

■ 순차 다이어그램의 구성요소

• 재귀 메시지

- 수신 객체가 자신의 메서드를 호출할 때 사용



• 답신 메시지

- 호출 메서드의 결과를 반환할 때 사용한다. 점선과 속이 채워진 삼각형으로 나타냄



■ 순차 다이어그램의 작성 과정

• 시나리오 정의

- 수정사항이 많이 발생할 우려가 있으므로 시나리오를 명확히 정의한 후에 그것을 토대로 순차 다이어그램을 작성

• 다이어그램과 시나리오의 1 : 1 대응

- 하나의 이벤트에 여러 개의 시나리오가 나올 경우 시나리오마다 각각의 다이어그램으로 표현

• 시나리오 분할

- 하나의 이벤트가 너무 길면 시나리오를 여러 개로 나누어 작성하는 것이 좋음

04. 순차 다이어그램

■ 순차 다이어그램

▪ 순차 다이어그램의 작성 과정

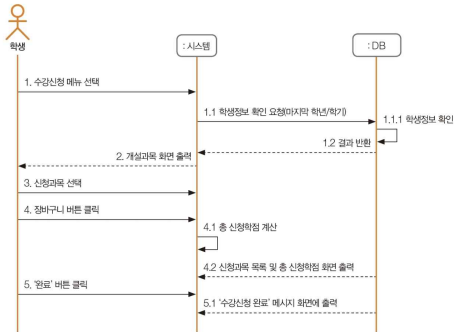


그림 2-28 '수강신청' 순차 다이어그램

05. 통신 다이어그램

■ 통신 다이어그램

■ 통신 다이어그램

- 객체 간 상호작용 관계에 주목
- 화살표 위에 적힌 번호로 순서를 알 수 있음
- 링크
 - 객체 간에 메시지를 주고받는 관계
 - 통신 다이어그램에서는 링크를 사용해 객체 간의 관계를 표현

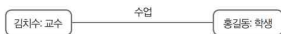


그림 2-29 링크로 객체 간의 관계 표현



그림 2-30 '수강신청' 통신 다이어그램

06. 활동 다이어그램

■ 활동 다이어그램

■ 활동 다이어그램

- 흐름도와 비슷하나 객체의 행위를 나타낸다는 점에서 다름
- 상위 수준에서는 업무의 흐름을 표현하고 분석 단계에서는 유스케이스의 구체적인 흐름(이벤트 흐름)을 나타냄
- 설계 단계에서는 클래스 내부 동작에 대한 알고리즘이나 구체적인 로직을 표현

■ 활동 다이어그램의 구성요소

• 시작/종료 상태, 활동

- 시작점: 활동의 시작을 나타내며 속이 채워진 원으로 표기
- 종료점: 활동의 종료를 나타내며 이중 원으로 표기
- 활동: 일의 처리와 실행을 나타내며 모서리가 둥근 사각형으로 표기
- 전이: 활동 간의 이동을 나타내며 화살표로 표기



그림 2-31 시작/종료 상태와 활동 기호

06. 활동 다이어그램

■ 활동 다이어그램

■ 활동 다이어그램의 구성요소

• 분기와 병합

- 분기: 조건에 의해 두 가지 경로로 나뉘는 위치이며 마름모를 사용 / 조건문은 마름모 옆에 << >> 기호로 작성
- 병합: 분기되어 각 활동을 수행하다가 합쳐지는 위치를 나타내며 분기와 같이 마름모를 사용 / 생략이 가능

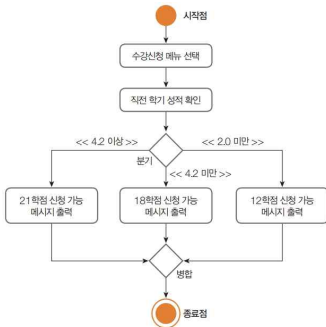


그림 2-32 분기와 병합

06. 활동 다이어그램

■ 활동 다이어그램

■ 활동 다이어그램의 구성요소

• 동기화 막대

- 여러 활동을 병행할 때 사용하는 것으로 동시 처리의 시작과 끝을 나타냄

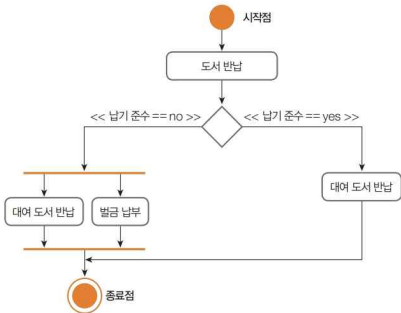


그림 2-33 동기화 막대

06. 활동 다이어그램

■ 활동 다이어그램

■ 활동 다이어그램의 구성요소

• 신호

- 활동 사이의 거래는 제어 신호를 보내는 방식으로 이루어짐

표 2-4 신호의 종류

종류	의미	표기법
행위 호출 call action	활동 activity, 동작 behavior, 오퍼레이션을 호출	
신호 송신 send signal	신호 행위를 기다리고 수신	
이벤트 수신 accept event	이벤트 행위를 기다리고 수신	
시간 이벤트 수신 accept time event	시간 이벤트 행위를 기다리고 수신	

• 구획면

- 수영장의 레인을 의미하는데 수영 선수가 자신의 레인을 넘어가면 안 되는 것처럼 해당 레인을 책임지는 객체가 있음

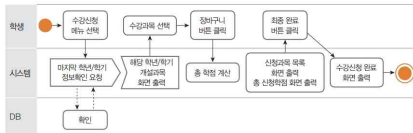


그림 2-34 '수강신청' 활동 다이어그램

07. 상태 다이어그램

■ 상태 다이어그램

■ 상태 다이어그램

- 이벤트 발생이나 시간의 흐름에 따라 바뀌는 객체의 상태를 나타냄

■ 상태 다이어그램의 구성요소

• 상태

- 객체가 존재할 수 있는 조건
- 객체의 존재 가능한 모든 상태가 파악되어야 함
- 상태는 모서리가 둥근 사각형으로 표현

• 전이

- 객체의 상태가 바뀌는 것을 나타내며 화살표를 사용해 표현

• 이벤트

- 객체의 상태를 바뀌게 하는 자극
- 화살표로 나타난 전이 위에 << >> 기호를 사용해 작성

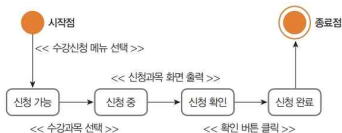


그림 2-35 상태 다이어그램

08. 컴포넌트 다이어그램

■ 컴포넌트 다이어그램

■ 컴포넌트 다이어그램

- 구현 관점에서 정적 모델링을 할 때 사용하는 것
- 어떤 실행 모듈이 존재하고 이들이 서로 어떤 연관성이 있는지의 종속 관계를 나타냄
- 사용자에게 논리적 또는 물리적 시스템의 구조를 볼 수 있게 해줌

■ 컴포넌트 다이어그램의 구성요소

• 컴포넌트

- 컴포넌트는 시스템을 구성하는 물리적인 요소로 프로그램 코드를 포함
 - <<executable>>: 실행 파일(.exe)
 - <<library>>: DLL 같은 정적 또는 동적 객체 라이브러리
 - <<table>>: 데이터베이스 테이블
 - <<file>>: 원시 코드를 포함하는 파일



그림 2-36 컴포넌트 표기

• 인터페이스

- 클래스 모양으로 나타내거나 아이콘을 사용해 간략하게 표현할 수 있음



(a) 클래스 모양으로 표기

그림 2-37 인터페이스 표기



(b) 아이콘으로 간략히 표기

08. 컴포넌트 다이어그램

■ 컴포넌트 다이어그램

■ 컴포넌트 다이어그램의 구성요소

- 의존 관계

- 서로 영향을 주고받는 관계를 말함
- 컴포넌트 간 연결은 점선 화살표로 나타내며 의존하는 쪽으로 화살표가 연결



그림 2-38 의존 관계

08. 컴포넌트 다이어그램

■ 컴포넌트 다이어그램

- 컴포넌트 다이어그램의 작성 과정
 - 컴포넌트 대상 정의
 - 무엇을 컴포넌트로 할지 정함
 - 컴포넌트 찾기
 - 컴포넌트 다이어그램에 포함할 컴포넌트를 식별
 - 컴포넌트 배치
 - 컴포넌트 다이어그램에 컴포넌트를 배치하고 이름을 붙여줌
 - 필요하다면 인터페이스를 정의하고 컴포넌트와 실현 관계로 연결
 - 의존 관계 정의
 - 컴포넌트 간 의존 관계를 정의

08. 컴포넌트 다이어그램

■ 컴포넌트 다이어그램

■ 컴포넌트 다이어그램의 작성 과정

- 각 컴포넌트의 인터페이스를 원으로 표시하고 인터페이스에 대한 실현은 컴포넌트와 연결

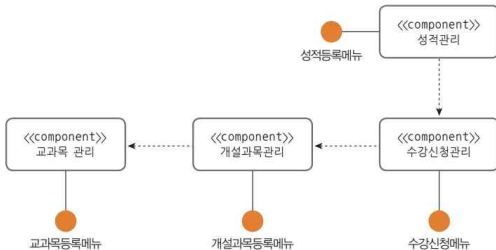


그림 2-39 컴포넌트 다이어그램

09. 배치 다이어그램

■ 배치 다이어그램

■ 배치 다이어그램

- 하드웨어 자원을 명시적으로 정의해 시스템의 물리적인 요소를 모델링하고 노드 간의 관계를 나타냄
- 노드: 시스템을 구성하는 처리 장치로 실행 파일 수준의 컴포넌트가 탑재된 하드웨어를 말함
- 노드를 연결하는 관계는 다양한 통신 방식을 의미

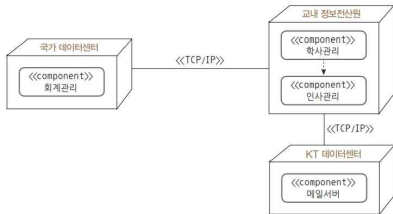


그림 2-40 배치 다이어그램

- 학사관리 서버는 교내 정보전산원에 설치
- 인사관리 서버는 교내 정보전산원에 설치
- 회계관리 컴포넌트는 국립대학교 전체가 동일한 것을 사용하고 서버는 국가 데이터센터에 배치하는 것으로 간주
- 메일 서버는 교외의 데이터센터에 설치

수고 많으셨습니다. 감사합니다.